ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

CI/CD Observability in Containerized Environments

Eduardo Sousa

Dissertação para obtenção do Grau de Mestre em Engenharia Informática, Área de Especialização em Cybersecurity And Systems Administration

Orientador: Dr. Alexandre Bragança
Supervisor: Dr. Joao Silva

Porto, January 7, 2026

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, Porto, January 7, 2026

Resumo

Containerized Continuous Integration / Continuous Delivery (CI/CD) platforms deployed on-premise with non-privileged access present critical observability challenges. Organizations managing infrastructure without host administrative privileges cannot deploy standard monitoring solutions that assume privileged container execution or cloud-managed telemetry. This research investigates container-first observability approaches for heterogeneous environments where conventional instrumentation is unavailable. Through systematic literature review following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) methodology, the work identifies a gap between cloud-native observability assumptions and operational realities in privilege-restricted deployments. The research designs and evaluates an integrated observability framework collecting metrics, logs and distributed traces from non-privileged containers across mixed Linux and Windows hosts. The solution employs sidecar exporters, agent collectors and runtime APIs to instrument CI/CD services without modifying container images or requiring host-level access. Empirical evaluation measures Mean Time To Detection (MTTD), Mean Time To Recovery (MTTR), telemetry pipeline throughput and instrumentation overhead against a defined baseline using controlled fault injection. The primary contributions include a reproducible reference architecture, prototype implementation with deployment automation, and practical guidance for constrained environments. The work provides evidence-based assessment of whether container-first instrumentation can measurably improve fault detection and diagnosis while maintaining resource overhead below operationally acceptable thresholds in privilege-restricted CI/CD platforms. **Palavras-chave:** CI/CD, Observability, Containers, Tracing, Logging, Monitoring

Resumo

Plataformas CI/CD contentorizadas implementadas on-premise com acesso não privilegiado apresentam desafios críticos de observabilidade. Organizações que gerem infraestruturas sem privilégios administrativos ao nível do anfitrião não conseguem implementar soluções de monitorização padrão que assumem execução privilegiada de contentores ou telemetria gerida em cloud. Esta investigação estuda abordagens de observabilidade centradas em contentores para ambientes heterogéneos onde a instrumentação convencional não está disponível. Através de revisão sistemática da literatura seguindo a metodologia PRISMA, o trabalho identifica uma lacuna entre os pressupostos de observabilidade cloud-native e as realidades operacionais em implementações com restrição de privilégios. A investigação concebe e avalia uma framework integrada de observabilidade que recolhe métricas, logs e traces distribuídos a partir de contentores não privilegiados em anfitriões mistos Linux e Windows. A solução emprega exportadores sidecar, coletores de agentes e APIs de runtime para instrumentar serviços CI/CD sem modificar imagens de contentores ou requerer acesso ao nível do anfitrião. A avaliação empírica mede MTTD, MTTR, débito do pipeline de telemetria e sobrecarga de instrumentação contra uma baseline definida usando injeção controlada de falhas. As principais contribuições incluem uma arquitetura de referência reproduzível, implementação de protótipo com automação de implementação, e orientação prática para ambientes com restrições. O trabalho fornece uma avaliação baseada em evidência sobre se a instrumentação centrada em contentores pode melhorar mensuravelmente a deteção e diagnóstico de falhas mantendo a sobrecarga de recursos abaixo de limiares operacionalmente aceitáveis em plataformas CI/CD com restrição de privilégios. **Palavras-chave:** CI/CD,

Observability, Containers, Tracing, Logging, Monitoring

Contents

Resumo **vii**

List of Figures **xi**

List of Tables **xiii**

List of Acronyms **xv**

1 Introduction **1**
1.1 Introduction and Problem Framing . . . 1
1.2 Context and Motivation . . . 2
1.2.1 Theoretical Context . . . 2
1.2.2 Organizational Reality . . . 2
1.3 Problem Statement . . . 3
1.3.1 Technical Problem . . . 3
1.3.2 Organizational Problem . . . 3
1.3.3 Scope and Boundaries . . . 4
1.4 Intended Contributions . . . 4
1.5 Research Objectives . . . 4
1.6 Research Questions . . . 5
1.7 Methodology Overview . . . 5
1.8 Ethical Considerations . . . 6
1.8.1 AI Usage Disclosure . . . 7
1.9 Document Structure . . . 7

2 Literature review **9**
2.1 PRISMA Methodology Overview . . . 9
2.1.1 Identification . . . 10
2.1.2 Screening . . . 10
2.1.3 Eligibility . . . 11
2.1.4 Inclusion and Data Extraction . . . 11
2.2 PRISMA Flow Diagram . . . 11
2.3 Instrumentation and Deployment Assumptions in the Literature . . . 12
2.4 Performance Evaluation Practices and Reported Overheads . . . 13
2.5 Discrepancies Between Research Environments and Constrained CI/CD Contexts . . . 14
2.6 Practitioner-Oriented Approaches and Industry Practices . . . 15
2.7 Identified Gaps and Implications for Constrained CI/CD Observability . . . 16
2.8 Mapping of Research Questions to Planned Research Activities . . . 17
2.8.1 RQ1: Non-Privileged Telemetry Collection . . . 18
2.8.2 RQ2: Cross-Server Correlation and Performance Trade-offs . . . 18

2.8.3 RQ3: Constraints, Trade-offs and Mitigation Strategies 19
2.9 Summary and Implications for Subsequent Research 19

3 Project Planning and Methodology **21**
3.1 Objectives and Scope Alignment 21
3.2 Methodological Approach 21
3.2.1 Planned Artifacts 21
3.2.2 Development Process and Traceability 22
3.2.3 Planned Evaluation Strategy 22
3.3 Skills Development Plan (PREPD) 23
3.3.1 Competency Diagnosis 23
3.4 Project Planning and Schedule 23
3.4.1 Phases, Activities, and Deliverables 24
3.4.2 Milestones and Dependencies 24
3.4.3 Gantt Chart 25
3.5 Project Monitoring and Management 25
3.6 Risk Management 26

4 Conclusion and Next Steps **29**
4.1 Synthesis of Preparation Work 29
4.2 Transition to Thesis Execution 29

Bibliography **31**

List of Figures

2.1 PRISMA flow diagram illustrating the systematic literature review process (Haddaway et al., 2022). The diagram shows the progression from initial identification through screening, eligibility assessment, and final inclusion stages, documenting the number of records at each stage and reasons for exclusion. 11

3.1 Project schedule overview (Gantt chart; January–June 2026) 25

List of Tables

3.1 Competency Diagnosis and Targets 23
3.2 Skills Development Actions Mapped to Project Phases 24
3.3 Planned Phases, Activities, and Deliverables (January–June 2026) 24
3.4 Planned Milestones and Acceptance Focus 25
3.5 Risk Rating Scales and Measurement Approach 26
3.6 Risk Register (technical, schedule, and external risks) 27

List of Acronyms

CI/CD	Continuous Integration / Continuous Delivery.
eBPF	extended Berkeley Packet Filter.
ISEP	Instituto Superior de Engenharia do Porto.
MTTD	Mean Time To Detection.
MTTR	Mean Time To Recovery.
PREPD	Preparation of Dissertation.
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses.

Chapter 1

Introduction

This chapter establishes the foundation for investigating observability frameworks in constrained containerized Continuous Integration / Continuous Delivery (CI/CD) environments. It frames the research problem within both theoretical and organizational contexts, articulates the specific challenges arising from non-privileged deployment constraints, and defines the scope and objectives that guide this work.

The chapter begins by presenting the operational challenges that motivate this research, then examines the gap between standard observability assumptions and the realities of organizations managing on-premise infrastructure without administrative privileges. Research questions, intended contributions, and methodological approach are subsequently outlined, providing a roadmap for the investigation documented in this preparation report.

1.1 Introduction and Problem Framing

Infrastructure failures in containerized CI/CD platforms create immediate operational challenges: determining which service failed, identifying root cause, and restoring functionality before development teams are blocked. In organizations running on-premise CI/CD infrastructure, diagnostic processes often require manual log inspection across multiple servers, timestamp correlation between disconnected systems, and inference from incomplete information. This fragmentation extends detection and resolution times, directly affecting development velocity and release schedules.

Standard monitoring solutions assume either full administrative access to host systems or cloud-managed platforms with integrated telemetry. Organizations managing containerized infrastructure without host administrative privileges fit neither model. They cannot deploy host-level monitoring agents, execute privileged containers, or modify container images of third-party services. Despite these constraints, operational visibility into service health, performance and fault conditions remains essential.

This project investigates observability approaches for containerized CI/CD platforms under non-privileged access constraints and heterogeneous infrastructure. The work examines patterns for collecting, correlating and analyzing telemetry when conventional monitoring deployment models are unavailable. The investigation focuses on a four-server on-premise deployment running mixed Linux and Windows hosts, where operations teams lack host administrative access and must instrument services without modifying container images.

The central research problem is: *How can an integrated observability framework be designed and evaluated to assess its potential to improve reliability and reduce incident response times under these constraints?*

1.2 Context and Motivation

The motivation for this research emerges from the intersection of theoretical understanding of observability frameworks and the operational realities encountered in specific organizational deployments. This section examines both the conceptual foundations established in software engineering research and the practical constraints that shape observability requirements in the target environment. The two subsections establish the theoretical basis for observability in CI/CD systems and describe the specific organizational context that defines the problem space for this investigation.

1.2.1 Theoretical Context

Continuous integration and continuous delivery (CI/CD) pipelines constitute critical infrastructure for software development organizations. Research in software engineering has established that pipeline reliability and observability directly influence development velocity, defect detection rates and time-to-market Forsgren et al., 2024. Modern CI/CD platforms typically comprise multiple specialized services: source control management (such as GitLab), build automation (such as Jenkins), static analysis (such as SonarQube), artifact management (such as Nexus) and security scanning (such as Dependency-Track). Each service generates distinct telemetry streams (metrics, logs, traces) that must be collected, correlated and analyzed to maintain operational visibility.

Observability frameworks follow established patterns: metrics for quantitative monitoring, structured logging for event analysis, and distributed tracing for transaction flow reconstruction. Cloud-native observability assumes elastic infrastructure, privileged host access and homogeneous execution environments, enabling instrumentation patterns such as node exporters with host access, kernel-level tracing and infrastructure-as-code deployment.

1.2.2 Organizational Reality

The organization where this project is developed manages on-premise CI/CD infrastructure serving multiple development teams across different business units. The platform runs on four physical servers: three Linux hosts and one Windows Server 2019 host. All CI/CD services operate as Docker Merkel, 2014 containers to facilitate deployment and isolation.

Two organizational constraints fundamentally shape the problem space. First, operations teams lack administrative privileges on host systems. Infrastructure administration is centralized under a separate IT department following corporate security policies, preventing installation of host-level monitoring agents, kernel modules or system-wide instrumentation. Second, CI/CD service containers must run without elevated privileges due to corporate security policy prohibiting privileged container execution.

These constraints create a gap between operational needs and available deployment options. The organization currently relies on application-level logs accessed through Docker CLI commands, manual inspection of container resource usage and reactive troubleshooting after user-reported incidents. This reactive posture results in prolonged incident detection (hours rather than minutes) and diagnosis cycles requiring coordinated investigation across disconnected log sources.

The practical impact manifests in three ways: delayed detection of service degradation until user complaints escalate, extended root cause analysis requiring manual correlation of logs

from five or more services, and inability to establish proactive capacity planning due to lack of historical resource utilization data. Development teams experience unpredictable pipeline failures attributed to infrastructure issues that operations teams cannot rapidly diagnose or prevent.

1.3 Problem Statement

The problem addressed in this research comprises interrelated technical and organizational dimensions that collectively define the observability challenge in constrained CI/CD environments. This section structures the problem statement across three complementary perspectives: the technical instrumentation challenges arising from privilege restrictions and platform heterogeneity, the organizational impact manifested through operational incidents and diagnostic delays, and the explicit boundaries that define what falls within and outside the scope of this investigation.

1.3.1 Technical Problem

Containerized CI/CD platforms deployed on-premise with non-privileged container execution and mixed operating systems present observability challenges requiring careful investigation. The technical problem comprises three interconnected dimensions:

1. **Instrumentation constraints:** Non-privileged containers cannot access host-level metrics (CPU, memory, disk, network at host scope), kernel instrumentation or system-wide performance counters. Container-visible metrics provide limited visibility into resource contention and host-level saturation.
2. **Telemetry fragmentation:** Each CI/CD service generates independent log streams in different formats. Without centralized collection and correlation, incident diagnosis requires sequential examination of logs from GitLab, Jenkins, SonarQube, Nexus and Dependency-Track through individual Docker CLI sessions.
3. **Cross-platform heterogeneity:** Linux and Windows containers exhibit different runtime APIs, log collection mechanisms and performance metric exposure. Instrumentation patterns must accommodate platform-specific differences while maintaining unified telemetry schemas.

1.3.2 Organizational Problem

The organization experiences concrete operational difficulties from limited observability. Recent incidents illustrate the problem severity:

- A GitLab database connection pool exhaustion incident in November 2025 was detected 3.5 hours after initial symptoms when multiple development teams reported failed pipeline executions. Root cause analysis required manual correlation of GitLab application logs, PostgreSQL (a relational database management system) container logs and Docker runtime inspection across two hosts.
- Intermittent Jenkins build failures in October 2025 attributed to insufficient executor capacity were diagnosed through trial-and-error resource adjustments over multiple days. Historical resource utilization data was unavailable to inform capacity planning decisions.

- A disk space saturation event in September 2025 affecting the Nexus artifact repository caused cascading build failures. Detection occurred only after builds began failing with cryptic upload errors. The absence of proactive disk usage monitoring prevented preventive action.

These incidents share common characteristics: detection relies on user reports rather than automated alerting, diagnosis requires time-consuming manual investigation without correlation tools, and preventive measures are hindered by lack of historical operational data. The organization estimates average Mean Time To Detection (MTTD) of 2–4 hours and Mean Time To Recovery (MTTR) of 4–8 hours for infrastructure-related incidents.

1.3.3 Scope and Boundaries

This research is scoped to non-privileged containerized environments with mixed operating systems (Linux and Windows). The investigation examines container-first observability patterns and does not address scenarios where host-level access or privileged execution is available. The work focuses on CI/CD platform services (source control, build automation, static analysis) rather than general-purpose application monitoring. Evaluation will be conducted within a single organizational context; generalizability to other constraint patterns and organizational settings remains to be established through future research.

1.4 Intended Contributions

The primary contributions planned are:

- A container-first reference architecture specifying patterns for collecting metrics, logs and traces from non-privileged containers in mixed OS on-premise environments, documenting component placement, data flows and correlation mechanisms.
- A reproducible prototype demonstrating feasible instrumentation patterns (sidecars, collectors, log forwarding) that operate without host administrative privileges, delivered as container images and deployment templates.
- An empirical evaluation measuring changes in MTTD, MTTR, pipeline throughput and instrumentation overhead, including documented measurement procedures for reproducibility and independent verification.
- Practical guidance and configuration artifacts to support adoption by operations teams constrained by privilege restrictions, validated through practitioner feedback.

1.5 Research Objectives

The objectives guiding this project are:

- **O1:** Design and evaluate a container-first reference architecture deployable without host administrative privileges, including component placement decisions and data flow patterns.
- **O2:** Develop a reproducible prototype of container images and deployment templates instrumenting representative CI/CD services, with documented configuration procedures and deployment automation.

- **O3:** Design and execute empirical evaluation comparing MTTD, MTTR, telemetry pipeline throughput and instrumentation overhead. Document measurement procedures, baseline collection methods and fault injection protocols for reproducibility.
- **O4:** Characterize resource overhead through CPU, memory and network utilization measurement. Document trade-offs between telemetry granularity and resource consumption, assessing whether instrumentation overhead can be maintained below 5%.
- **O5:** Provide configuration templates, deployment scripts and documented procedures for constrained on-premise environments, including decision criteria for tool selection and deployment patterns, validated through practitioner feedback.

Each objective is defined with measurable outcomes and explicit deliverables. Objective O3 includes concrete measurement procedures (baseline collection, controlled fault injection, paired comparisons) to support quantitative assessment. Objective O4 establishes an explicit threshold (5% overhead target) to guide evaluation and inform adoption decisions; the research will assess whether this threshold can be achieved in practice.

1.6 Research Questions

This project addresses the following questions:

1. How can metrics, logs and distributed traces be collected from containerized CI/CD services when only non-privileged container execution is permitted?
2. Which instrumentation and deployment patterns enable effective cross-server correlation in mixed OS environments while maintaining acceptable resource overhead?
3. What practical constraints and trade-offs are introduced by non-privileged telemetry collection, and how can these be quantified and mitigated within a reproducible reference architecture?

1.7 Methodology Overview

This project follows a design science research approach Hevner et al., 2004, combining systematic literature review, comparative technology assessment, artifact design, prototype implementation and empirical evaluation. Design science is appropriate because it addresses a practical problem through creation and evaluation of a novel artifact (the observability framework) in a specific organizational context. The research integrates qualitative methods (literature synthesis, practitioner interviews) with quantitative methods (performance measurement, resource overhead characterization). The work is organized into five phases:

Phase 1 (Problem Analysis and Literature Review): A PRISMA-guided systematic review examines observability frameworks, containerized monitoring and CI/CD reliability literature to identify existing approaches, documented gaps and constraint patterns relevant to non-privileged deployment contexts.

Phase 2 (Technology Assessment): Evaluation of candidate tools (including Prometheus Volz, 2016 for metrics, Grafana Loki for logs, Jaeger for distributed tracing, and OpenTelemetry for instrumentation) against defined criteria: non-privileged execution capability, cross-platform support (Linux and Windows), resource overhead characteristics and integration complexity. Assessment produces a justified tool selection with documented trade-off analysis.

Phase 3 (Architecture Design): Specification of component placement (exporters, collectors, storage, visualization), data flow patterns and correlation mechanisms. Deliverables include deployment diagrams, configuration templates and documented design decisions with rationale.

Phase 4 (Prototype Implementation): Implementation and documentation of a reproducible prototype using selected instrumentation patterns. Deliverables include deployment automation scripts, configuration artifacts and operational documentation.

Phase 5 (Evaluation and Validation): Quantitative evaluation of MTTD, MTTR and instrumentation overhead through controlled fault-injection experiments with documented baseline measurements. Qualitative validation through structured practitioner interviews captures operational acceptability and deployment constraints.

The methodology balances artifact creation (reference architecture and prototype) with empirical validation (quantitative metrics and qualitative feedback). Quantitative evaluation establishes measurable outcomes; qualitative data addresses operational acceptability and deployment considerations that quantitative metrics alone cannot capture.

1.8 Ethical Considerations

This work adheres to the ethical guidelines established by Instituto Superior de Engenharia do Porto (Instituto Superior de Engenharia do Porto (ISEP)) and the IEEE Code of Ethics ("IEEE Xplore Full-Text PDF:", n.d.). The research, analysis, experimental design, and all scientific contributions presented in this document are authored by the student. The student assumes full responsibility for the integrity, accuracy and validity of all research findings, interpretations and conclusions.

This research involves human subjects activities (structured practitioner interviews) and access to operational telemetry data. All activities will follow institutional ethical review procedures. Ethical clearance will be obtained before commencing interviews or data collection. Participants will provide documented informed consent and will be informed of their right to withdraw. Interview data and operational telemetry will be anonymized to remove identifying information. Data handling will conform to institutional policies and applicable data protection regulations, including GDPR where relevant.

Production deployments will be preceded by staging environment validation and change management approval to minimize operational disruption risk. The project observes data minimization principles: telemetry collection is limited to technical metrics necessary for evaluation objectives. Personal data collection, if required, will be explicitly justified, documented and subject to institutional review board approval.

When presenting empirical results, host identifiers, network topology details and sensitive configuration information will be anonymized to protect organizational confidentiality. Access to production systems and telemetry data will be governed by organizational access

control policies. All collected data will be stored securely and retained only for the duration necessary to complete the research, after which it will be securely disposed of in accordance with institutional data retention policies.

1.8.1 AI Usage Disclosure

Artificial intelligence tools were employed in a limited supporting capacity during the preparation of this work. AI assistance was restricted to: generating search keyword suggestions to support systematic literature identification; assisting with title and abstract screening during PRISMA-guided literature review; offering alternative phrasings for sentence reformulation to improve clarity and readability; and performing spell checking and grammar verification. AI tools were not used to generate scientific arguments, formulate research contributions, design experiments, analyze results, or draw conclusions. All intellectual content, methodological decisions, and scientific judgments remain the original work of the author.

1.9 Document Structure

This preparation report documents the research plan, including problem framing, literature review, and planned methodology. The remainder of this document is organized as follows: Chapter 2 presents the systematic literature review following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) methodology, synthesizing existing observability research and identifying gaps relevant to constrained CI/CD environments; Chapter 3 describes project planning, methodology details, skill management, resource allocation and the execution timeline; Chapter 4 summarizes the work presented and outlines next steps.

Chapter 2

Literature review

This chapter presents a systematic review of observability frameworks for containerized systems, with particular focus on instrumentation techniques, deployment constraints, and performance characteristics relevant to CI/CD environments. Following PRISMA guidelines, the review synthesizes academic literature and industry practices to establish the current state of knowledge on telemetry collection under privilege restrictions, cross-platform deployment patterns, and empirical overhead evaluation methodologies.

The synthesis examines how existing observability approaches address the operational realities of non-privileged containerized infrastructure, heterogeneous execution environments, and ephemeral task-based workloads. By identifying documented privilege assumptions, performance trade-offs, and methodological gaps in the literature, this chapter establishes the foundation for subsequent architecture design, prototype implementation, and empirical evaluation aligned with the research questions defined in Chapter 1.

2.1 PRISMA Methodology Overview

A systematic literature review was conducted following PRISMA guidelines to ensure comprehensive and reproducible coverage of the research domain. This section describes the methodological structure applied to identify, screen, assess, and include relevant literature. The subsections that follow document each stage of the PRISMA process: identification of candidate sources through database queries, screening based on defined inclusion and exclusion criteria, eligibility assessment through full-text review, and final inclusion with structured data extraction.

The search strategy includes:

Information sources:

- **Academic databases:** IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar for peer-reviewed publications
- **Technical repositories:** arXiv for preprints and working papers in systems and software engineering
- **Gray literature:** Technical documentation from established open-source observability projects, industry whitepapers, and practitioner reports from reputable sources (given the rapid evolution of container observability practices)

Search terms and queries: Core search queries will combine terms from three concept groups using Boolean operators:

1. *Observability concepts:* "observability", "monitoring", "telemetry", "instrumentation", "metrics", "logging", "tracing", "distributed tracing"
2. *Container and deployment context:* "container", "containerized", "Docker", "non-privileged", "unprivileged", "sidecar", "agent"
3. *Application domain:* "CI/CD", "continuous integration", "continuous deployment", "DevOps", "reliability", "incident response", "MTTD", "MTTR"

Example query structure: (*observability* OR *monitoring* OR *telemetry*) AND (*container* OR *containerized* OR *non-privileged*) AND (*CI/CD* OR *reliability* OR *incident response*)

Inclusion and exclusion criteria: *Inclusion:* Peer-reviewed publications (2018 to 2025), technical reports on container telemetry, instrumentation patterns, performance evaluation, reliability improvement in constrained environments, comparative observability studies, and empirical overhead evaluations.

Exclusion: Publications requiring privileged host access without constraint discussion, cloud-provider-specific solutions not generalizing to on-premise infrastructure, purely theoretical work lacking practical guidance, papers before 2018 (except seminal works).

Selection process: Literature selection followed a three-stage process: (1) title and abstract screening to identify potentially relevant sources, (2) full-text review to assess relevance against inclusion/exclusion criteria, and (3) quality assessment focusing on methodological rigor, reproducibility, and applicability to constrained environments. Reference chaining (backward and forward citation tracking) supplemented database searches to identify additional relevant work.

Gray literature and industry sources: In addition to peer-reviewed academic literature, industry documentation (including technical whitepapers, platform documentation from established observability projects such as OpenTelemetry, Prometheus, and Grafana, as well as vendor technical reports) was identified through targeted web searches and documentation repositories. These sources were assessed for technical credibility, relevance to practitioner-oriented deployment patterns, and alignment with the operational constraints described above. Industry sources complement the empirical evidence base by documenting real-world instrumentation practices, configuration tradeoffs, and platform capabilities, though they were not subjected to the same formal quality assessment criteria applied to peer-reviewed publications. These materials inform the synthesis of practitioner-oriented approaches (Section 2.6) but are distinguished from the core empirical studies that form the primary evidence base.

2.1.1 Identification

The identification stage recorded search queries, databases queried, and the number of retrieved records. Queries were versioned and documented to enable replication of the search process.

2.1.2 Screening

Title and abstract screening was performed using documented inclusion/exclusion criteria; screening decisions and reasons for exclusion at this stage were recorded. When uncertain, items were advanced to fulltext review to avoid premature exclusion.

2.1.3 Eligibility

Fulltext review assessed whether each study met the inclusion criteria and whether it reported sufficient methodological detail for quality assessment. Studies that lacked sufficient detail were noted but excluded from quantitative synthesis.

2.1.4 Inclusion and Data Extraction

Data from included studies were extracted into structured evidence tables. Extraction fields included: study context, deployment environment (cloud/on-premise), privilege assumptions, telemetry modalities used, evaluation metrics and experimental design. Extraction was validated for accuracy and completeness.

2.2 PRISMA Flow Diagram

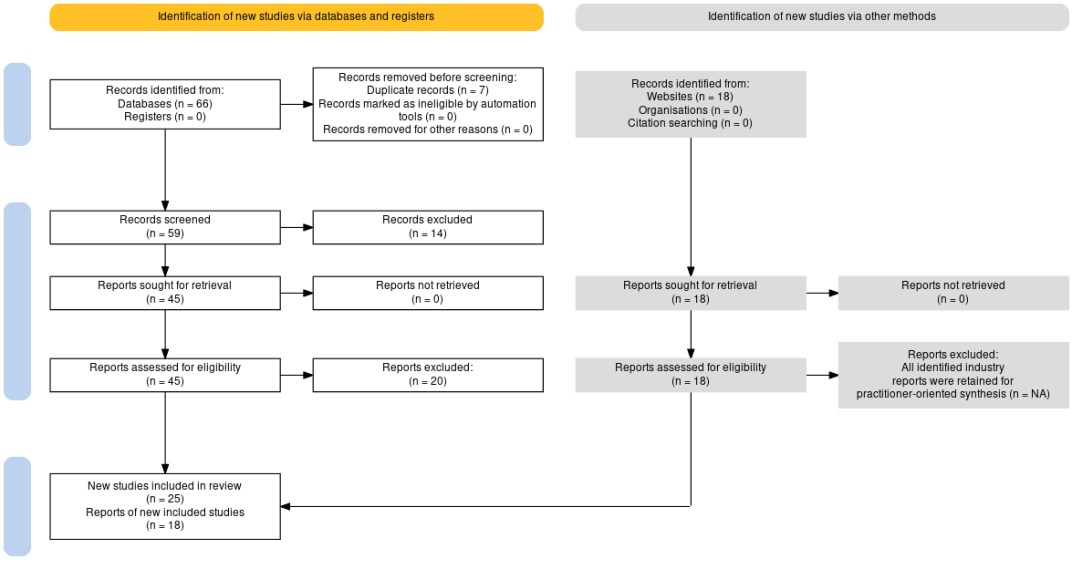

Figure 2.1: PRISMA flow diagram illustrating the systematic literature review process (Haddaway et al., 2022). The diagram shows the progression from initial identification through screening, eligibility assessment, and final inclusion stages, documenting the number of records at each stage and reasons for exclusion.

The systematic literature review process, as depicted in Figure 2.1, followed the four stages defined by the PRISMA 2020 protocol. The identification stage retrieved sixty-six records from academic databases (IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar). In parallel, eighteen additional records were identified through targeted searches of practitioner-oriented sources, including observability tooling documentation and CI/CD platform vendor materials. No records were identified from trial registers.

During deduplication and initial cleaning, seven records were removed due to duplication or because they did not constitute valid research publications (e.g., documentation links or malformed entries). The screening stage therefore evaluated fifty-nine records through title and abstract review against predefined inclusion and exclusion criteria focused on container

observability, telemetry collection techniques, and relevance to constrained deployment environments. This screening resulted in the exclusion of fourteen records that were out of scope or lacked sufficient relevance.

The eligibility stage assessed forty-five reports through full-text review, examining methodological rigor, privilege assumptions, workload characteristics, and applicability to privilege-restricted CI/CD contexts. No reports were excluded due to retrieval issues. Following eligibility assessment, twenty-five peer-reviewed academic studies met all inclusion criteria and were retained as the core evidence base for the critical analysis presented in subsequent sections.

In addition to the academic literature, eighteen industry sources were retained as complementary evidence to document operational deployment patterns and platform capabilities. These practitioner-oriented sources were included as gray literature to contextualize real-world practices but were not subjected to the same formal quality assessment criteria applied to peer-reviewed studies.

The twenty-five peer-reviewed academic studies that form the core evidence base of this systematic review are collectively cited here to ensure complete traceability between the PRISMA inclusion count and the bibliography (Correia et al., 2023; Dinga et al., 2023; Großmann & Klug, 2017; Hasanth & Nadig, 2024; "Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems", n.d.; Janecek et al., 2021; Kannan et al., 2024; Karkan, n.d.; Levin, 2020; Liu et al., 2019; Norgren, n.d.; Pappula et al., 2021; Pi et al., 2018; Raith et al., 2022; Ramachandran et al., 2023; Sahu et al., 2023; Salcedo-Navarro et al., 2025; Sandberg, n.d.; Sharma, 2023; Sharma & Nadig, 2024; Soldani et al., 2023; Tan et al., 2023; Taylor et al., 2020; "TraceWeaver", n.d.; Usman et al., 2023). While subsequent sections analyze a subset of these studies in detail,particularly those addressing instrumentation techniques, performance overhead, and deployment constraints, the remaining works inform background assumptions, contextual framing, and the identification of gaps relevant to constrained CI/CD observability.

2.3 Instrumentation and Deployment Assumptions in the Literature

The literature synthesis documents privilege-dependent observability features and evaluates the deployment assumptions underpinning reviewed systems (Section 2.3).

The comparative analysis of published observability systems demonstrates a consistent pattern of deployment and privilege assumptions that influence the applicability of proposed solutions to constrained environments. A substantial proportion of evaluated architectures presume host-level administrative access, either through privileged containers with direct socket access, kernel-level instrumentation frameworks, or cluster-wide operator deployments. Correira et al. Correia et al., 2023 exemplify this pattern with a hybrid push-pull architecture requiring node-level agents with `cgroup` (Linux control groups, a kernel feature for resource isolation and accounting) access for container metrics and RAPL (Running Average Power Limit, an Intel interface for energy monitoring) interfaces for energy profiling, capabilities unavailable without elevated privileges. Similarly, Dinga et al. Dinga et al., 2023 deploy host-based collectors such as Metricbeat (an Elastic agent for shipping system and service metrics) and cAdvisor (Container Advisor, a tool for analyzing container resource usage) with direct Docker API access, supplemented by application-level Zipkin

(a distributed tracing system) tracing that necessitates explicit integration with containerized services. These deployment patterns, while effective in homogeneous cloud platforms where orchestrators manage infrastructure, present significant barriers in heterogeneous or restricted environments where node access is administratively controlled.

A second class of instrumentation approaches leverages kernel observability mechanisms, particularly extended Berkeley Packet Filter (eBPF), to achieve non-intrusive telemetry collection. Works such as Sharma et al. Sharma and Nadig, 2024 and Kannan et al. Kannan et al., 2024 demonstrate that kernel-level probes can extract rich observability signals (including network flow metrics, system call traces, and container-level resource utilization) without requiring application code modifications. However, these mechanisms impose their own set of deployment prerequisites: Linux kernel versions that support stable eBPF interfaces (typically v5.15 or newer), capabilities such as `CAP_BPF` (a Linux capability that grants permission to use eBPF) and `CAP_NET_ADMIN` (a Linux capability for network administration operations), and deployment as privileged DaemonSets (Kubernetes workloads that run one pod per node) with node-wide scope. While such techniques offer substantial reductions in instrumentation overhead compared to sidecar proxies or per-service agents, they remain inapplicable in scenarios where host kernel access is restricted or where execution occurs on heterogeneous platforms without uniform kernel feature availability. The distinction between operational feasibility and privilege requirements emerges as a critical dimension in assessing the transferability of observability architectures to environments lacking administrative control over underlying infrastructure.

Sidecar-based collection represents a third architectural family, evaluated extensively in service mesh (a dedicated infrastructure layer for handling service-to-service communication) contexts by Hasanth et al. Hasanth and Nadig, 2024 and Salcedo-Navarro et al. Salcedo-Navarro et al., 2025. Sidecars intercept network traffic at the pod boundary, enabling transparent telemetry collection without modifying application logic. Yet sidecar deployments introduce resource multipliers proportional to service count, where each sidecar consumes CPU, memory, and network bandwidth, and rely on orchestrator-managed injection mechanisms that may not be available or permissible in restrictive CI/CD environments. Furthermore, latency analysis by Sahu et al. Sahu et al., 2023 indicates measurable microarchitectural contention between application and sidecar containers sharing vCPU resources, underscoring that deployment simplicity does not eliminate performance implications.

2.4 Performance Evaluation Practices and Reported Overheads

Empirical evaluations in the reviewed literature exhibit considerable heterogeneity in experimental design, workload selection, and performance metrics, complicating direct comparisons across studies. Benchmark applications vary from synthetic microservice suites such as Online Boutique (a Google Cloud microservices demo application) Hasanth and Nadig, 2024 and DeathStarBench (a suite of microservice benchmarks) "TraceWeaver", n.d. to production-scale workloads deployed in commercial platforms Liu et al., 2019. Load generation techniques range from controlled concurrency ramps using tools such as Locust (a load testing tool) to stochastic arrival models with diverse traffic patterns. Overhead is quantified through disparate metrics, including CPU utilization percentages, absolute memory footprints, throughput degradation relative to baseline, and latency percentiles, reported at

different granularities (per-node, per-container, system-wide aggregates). This methodological diversity reflects the absence of standardized observability benchmarking protocols for containerized systems, a gap that hinders reproducibility and cross-study synthesis.

Reported overhead magnitudes span a wide range depending on instrumentation technique and signal fidelity. Host-based eBPF solutions demonstrate notably lower resource consumption: Sharma et al. Sharma and Nadig, 2024 report CPU usage reductions of twenty-one to two-hundred-fourteen times compared to OpenTelemetry agents, sidecar proxies, and traditional host exporters. Kannan et al. Kannan et al., 2024 document approximately ten percent throughput degradation and point-five percent per-node CPU overhead for network flow monitoring via Traffic Control hooks, contrasting favorably with legacy packet capture tools that impose fifty to seventy percent throughput penalties. Conversely, full-stack distributed tracing through export-based OpenTelemetry collectors incurs substantial costs: performance evaluations on serverless and microservice workloads reveal throughput reductions ranging from nineteen to eighty percent and latency increases between seven and one-hundred-seventy-five percent "Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems", n.d. Tail-based sampling strategies, while improving trace completeness, amplify collector CPU consumption by seventy-one percent relative to head-based sampling Karkan, n.d. These figures underscore the existence of fundamental tradeoffs between signal completeness, collection latency, and resource overhead, tradeoffs that must be navigated in resource-constrained deployment contexts.

Several studies acknowledge methodological limitations that restrict the generalizability of overhead assessments. Dinga et al. Dinga et al., 2023 note the inability to isolate container-level energy consumption using external power measurement, attributing observed increases to host system-wide effects rather than application-specific telemetry costs. Norgren Norgren, n.d. highlights that collector placement decisions interact with orchestrator resource limits and TraceContext propagation overhead in ways that are not easily predicted from component-level microbenchmarks. Ramachandran et al. Ramachandran et al., 2023 document latency amplifications between thirteen and forty-five times baseline when employing comprehensive eBPF syscall tracing for fault diagnosis, demonstrating that even low-overhead kernel mechanisms impose non-negligible penalties when probe invocation frequency is high. The variability of reported overheads reflects not only differences in experimental setup but also the sensitivity of performance outcomes to workload characteristics, instrumentation scope, and backend processing efficiency.

2.5 Discrepancies Between Research Environments and Constrained CI/CD Contexts

The operational assumptions underlying much of the published research diverge substantially from the constraints characteristic of on-premise CI/CD deployments in heterogeneous or administratively restricted environments. The majority of evaluated systems presume orchestrator-mediated infrastructure management, often Kubernetes Burns et al., 2016, with the authority to deploy cluster-wide resources such as DaemonSets, CustomResourceDefinitions (user-defined extensions to the Kubernetes API), and admission controllers (plugins that intercept API requests to enforce policies). This model affords centralized configuration, uniform telemetry pipelines, and the ability to inject sidecars or agents into workloads declaratively. By contrast, CI/CD systems operating on shared infrastructure with limited

user privileges cannot rely on such orchestrator capabilities. Executors may run as unprivileged containers without access to host namespaces, kernel interfaces, or Docker sockets, precluding the direct application of host-based or kernel-level instrumentation techniques documented in the literature.

Research deployments typically assume homogeneous execution environments (identical kernel versions, uniform container runtimes, consistent network plugins) facilitating predictable instrumentation behavior. Real-world CI/CD infrastructure, particularly in organizations with diverse legacy systems or multi-cloud deployments, exhibits platform heterogeneity: varying operating system distributions, mixed kernel versions, and differing container runtime policies. Observability mechanisms dependent on specific kernel features, such as eBPF tracepoints introduced in recent Linux releases, fail silently or degrade gracefully depending on fallback support, introducing operational fragility. The assumption of code modifiability presents another divergence: academic prototypes often demonstrate instrumentation by integrating tracing libraries into purpose-built microservice applications, whereas CI/CD pipelines frequently execute third-party packaged software (build tools, testing frameworks, dependency resolvers) that cannot be recompiled or instrumented without violating support contracts or introducing version drift.

A third area of misalignment concerns the evaluation of correlation mechanisms across observability signals. Research systems implementing distributed tracing commonly propagate trace context (metadata linking related operations across service boundaries) through HTTP headers or gRPC metadata Sigelman et al., 2010, mechanisms that require either application-level instrumentation or transparent proxy interception at service boundaries. In CI/CD contexts, correlation must often be reconstructed post hoc from logs, metrics, and execution metadata available through pipeline orchestration APIs, without the benefit of in-band trace propagation. This retrospective correlation problem (linking container resource spikes to specific job identifiers, associating log error patterns with pipeline failure events) remains underexplored in the literature, which tends to assume instrumentation frameworks designed for long-lived service deployments rather than ephemeral task-based workloads. The temporal and architectural characteristics of CI/CD executions (short-lived containers, stateless task execution, batch job semantics) differ meaningfully from the microservice interaction patterns that dominate the observability research corpus.

2.6 Practitioner-Oriented Approaches and Industry Practices

Beyond the academic literature, industry documentation and technical reports from observability platform vendors provide complementary perspectives on telemetry collection in production environments. These sources emphasize operational pragmatism, configuration flexibility, and the incremental adoption of observability capabilities within existing infrastructure constraints. For instance, OpenTelemetry project documentation (OpenTelemetry Authors, 2024) outlines collector deployment patterns ranging from agent-per-host configurations to centralized gateway architectures, explicitly addressing tradeoffs between local buffering, network bandwidth consumption, and backend ingestion capacity. Prometheus Volz, 2016 operational guidelines (Prometheus Authors, 2024) recommend pull-based metric collection with service discovery mechanisms that decouple telemetry exporters from backend topology, enabling declarative configuration suitable for dynamic containerized environments. Grafana Loki documentation (Grafana Labs, 2024) describes log aggregation without indexing full

message content, reducing storage overhead at the cost of query expressiveness, a tradeoff relevant to resource-constrained deployments.

Vendor technical whitepapers and case studies, while less methodologically rigorous than peer-reviewed research, document deployment experiences across diverse organizational contexts. These reports frequently highlight challenges not emphasized in academic work: authentication and authorization models for multi-tenant telemetry pipelines, data retention policies balancing compliance requirements against storage costs, and observability tool sprawl arising from heterogeneous technology stacks. Datadog (Datadog, 2023) and Dynatrace (Dynatrace, 2023) technical blogs discuss the operational complexity of maintaining unified observability across hybrid cloud and on-premise infrastructure, noting that privilege limitations in customer-managed environments necessitate agent deployment models that sacrifice telemetry completeness for installation simplicity. Elastic observability architecture documentation (Elastic, 2024) describes hybrid push-pull patterns where agents push metrics and logs to centralized endpoints, accommodating firewall traversal and egress-only network policies common in enterprise settings.

Industry practices also reveal the prevalence of logging-centric observability as a pragmatic compromise when comprehensive instrumentation is infeasible. Structured logging with correlation identifiers, combined with centralized log aggregation and pattern matching, provides limited but actionable visibility without requiring application code changes or privileged host access. Jenkins (Jenkins Project, 2024b) and GitLab (GitLab, 2024) CI/CD platform documentation emphasizes log-based troubleshooting workflows, reflecting the operational reality that pipeline executions are often diagnosed through console output rather than through distributed traces or granular metrics. While distributed tracing capabilities are available through plugins such as Jenkins OpenTelemetry (Jenkins Project, 2024a), they are not enabled by default and are not central to documented debugging workflows. This logging-first approach aligns with the ephemeral nature of CI/CD workloads and the difficulty of instrumenting arbitrary third-party build tools. However, the literature provides minimal empirical guidance on the diagnostic effectiveness of log-only observability compared to multi-signal approaches, leaving a gap between documented best practices and evidence-based performance validation.

2.7 Identified Gaps and Implications for Constrained CI/CD Observability

The synthesis of reviewed literature reveals several areas where existing research provides limited guidance for observability in privilege-constrained, heterogeneous CI/CD environments. First, empirical studies evaluating non-privileged telemetry collection patterns remain scarce. While sidecar-based approaches nominally operate without host-level privileges, performance evaluations typically occur in Kubernetes clusters where orchestrator-managed injection and resource allocation mitigate operational complexity. The feasibility and overhead characteristics of sidecar telemetry in environments where container orchestration is absent or restricted (such as standalone Docker executor environments or VM-based runners) have not been systematically characterized. Similarly, the applicability of application-level instrumentation libraries to third-party packaged software, a common scenario in CI/CD pipelines, receives minimal attention in the literature, which predominantly evaluates custom-built microservice applications amenable to source-level modification.

Second, the literature offers limited insight into post hoc correlation techniques suitable for ephemeral task-based workloads. Research on distributed tracing emphasizes in-band context propagation through service invocation chains, a model poorly suited to CI/CD executions where observability signals must be correlated retrospectively using pipeline metadata, container identifiers, and temporal alignment of logs and metrics. Techniques for reconstructing causal relationships between resource contention events and pipeline failures, or for attributing aggregate host metrics to individual containerized build tasks, are underexplored. This gap is significant because CI/CD observability fundamentally addresses a diagnostic problem (identifying which pipeline stage failed and why) rather than the performance optimization focus that dominates microservice observability research.

Third, the methodological practices documented in the literature provide insufficient guidance for evaluating observability interventions in controlled CI/CD settings. Standard microservice benchmarks such as Online Boutique or DeathStarBench model synchronous request-response patterns with stateless service replicas, characteristics that diverge from CI/CD workload semantics: batch job execution, stateful build caches, sequential pipeline stages with inter-stage dependencies. Metrics such as MTTD and MTTR, frequently cited as primary outcome measures for observability effectiveness, lack standardized operational definitions in the CI/CD context. Variability in how detection events are defined (alert trigger, operator awareness, automated remediation) and how recovery is measured (pipeline restart, job completion, service restoration) complicates the interpretation and comparison of reported improvements. The absence of reproducible benchmark artifacts (representative CI/CD workloads, fault injection scenarios, instrumentation templates) further hinders empirical validation of observability claims.

Finally, the literature exhibits limited engagement with operational constraints imposed by multi-tenant shared infrastructure, data residency requirements, and restricted network egress policies. Research prototypes typically assume unimpeded network connectivity to backend observability platforms, whereas enterprise CI/CD environments may mandate on-premise data retention, prohibit external telemetry export, or enforce egress filtering that complicates agent-to-collector communication. The implications of such constraints for collector topology, buffering strategies, and telemetry completeness remain largely unaddressed. These gaps collectively indicate that while the observability research community has produced substantial evidence on instrumentation techniques, performance tradeoffs, and architectural patterns, the direct applicability of these findings to constrained CI/CD contexts requires further investigation and adaptation.

2.8 Mapping of Research Questions to Planned Research Activities

This section clarifies how the research questions defined in Chapter 1 will be addressed through planned research activities following the Design Science Research methodology. The systematic review presented in this chapter provides the empirical and conceptual foundation by synthesizing existing observability research, identifying privilege assumptions and deployment constraints documented in the literature, and establishing where current evidence provides partial answers or reveals gaps that must be addressed through future artifact development and evaluation.

The discussion below distinguishes between what has been established through literature synthesis (state of the art) and what remains to be investigated through planned design,

implementation, and evaluation activities. Each subsection addresses one research question, documenting how the literature review informs the question, what gaps remain, and how future research phases will provide answers through architectural specification, prototype development, and empirical assessment.

2.8.1 RQ1: Non-Privileged Telemetry Collection

The first research question asks: *How can metrics, logs and distributed traces be collected from containerized CI/CD services when only non-privileged containers are available (no host administrative access)?*

The systematic review presented in this chapter synthesizes existing instrumentation techniques and documents their privilege requirements in detail (Section 2.3). The analysis reveals that most documented approaches in the literature presume host-level access, orchestrator-managed resource injection, or the ability to deploy privileged containers with elevated capabilities. For example, eBPF-based solutions (Kannan et al., 2024; Sharma & Nadig, 2024) require kernel access and capabilities such as `CAP_BPF`, while host-based exporters (Correia et al., 2023; Dinga et al., 2023) depend on Docker socket access or `cgroup` filesystem visibility. The gap analysis (Section 2.7) establishes that empirical studies evaluating non-privileged telemetry collection patterns specifically for CI/CD workloads remain scarce, and that sidecar evaluations predominantly occur in Kubernetes environments where orchestrator capabilities mitigate operational complexity.

Planned research activities: The next phase will involve architectural specification of container-first exporters and collectors that operate without host privileges, followed by prototype implementation demonstrating feasible instrumentation patterns for representative CI/CD services. Empirical evaluation will assess telemetry completeness, signal fidelity, and collection reliability under operational constraints. The evidence base established in this chapter will inform the selection of candidate technologies and deployment patterns compatible with the privilege restrictions documented throughout the literature.

2.8.2 RQ2: Cross-Server Correlation and Performance Trade-offs

The second research question asks: *Which instrumentation and deployment patterns (sidecars, agent collectors, push vs pull pipelines) best support cross-server correlation in mixed OS environments while maintaining acceptable resource overhead?*

The literature review examines sidecar architectures (Hasanth & Nadig, 2024; Salcedo-Navarro et al., 2025), push-pull telemetry pipelines, and correlation mechanisms documented in microservice observability contexts (Sections 2.3 and 2.4). The synthesis reports empirically measured performance overheads from existing studies: sidecar proxies introduce measurable latency and microarchitectural contention (Sahu et al., 2023), eBPF solutions demonstrate substantially lower CPU consumption compared to application-level instrumentation (Sharma & Nadig, 2024), and distributed tracing with full signal export can impose throughput reductions ranging from nineteen to eighty percent ("Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems", n.d.). However, the literature provides limited guidance on heterogeneous OS environments (mixed Linux and Windows hosts) and post hoc correlation techniques suitable for ephemeral CI/CD workloads, where trace context propagation Sigelman et al., 2010 through HTTP headers or service mesh mechanisms may not be applicable (Section 2.5).

Planned research activities: Future work will conduct comparative technology assessment to select deployment patterns that balance correlation effectiveness with resource constraints. This will be followed by design and implementation of topology and correlation mechanisms adapted to CI/CD execution semantics. Quantitative evaluation will measure correlation accuracy, telemetry pipeline throughput, and instrumentation overhead against the operational target of maintaining resource consumption below five percent where feasible. The performance baselines and trade-off characterizations documented in this chapter will provide reference points for comparative assessment.

2.8.3 RQ3: Constraints, Trade-offs and Mitigation Strategies

The third research question asks: *What are the practical constraints and trade-offs (data fidelity, retention, network and CPU overhead, security) introduced by non-privileged telemetry collection, and how can they be mitigated in a reproducible reference architecture?*

The literature synthesis documents privilege-dependent observability features and evaluates the deployment assumptions underpinning reviewed systems (Section 2.3). The analysis identifies that research prototypes typically presume capabilities unavailable in constrained environments: orchestrator-mediated infrastructure management, homogeneous execution platforms with uniform kernel versions, and the ability to modify or instrument application code (Section 2.5). The gap analysis (Section 2.7) highlights methodological limitations in existing evaluation practices, including the absence of standardized operational definitions for MTTD and MTTR in CI/CD contexts, limited guidance on data retention trade-offs under storage constraints, and the lack of reproducible benchmark artifacts suitable for controlled evaluation of constrained observability systems.

Planned research activities: The design and implementation phases will systematically document operational constraints encountered during architecture specification and prototype development. Planned evaluation activities will characterize trade-offs between telemetry completeness and resource overhead, and will provide configuration guidance addressing security boundaries, network egress restrictions, and multi-tenant isolation requirements. Empirical assessment will quantify data fidelity (signal loss rates, sampling effects), retention feasibility (storage growth, query performance), and overhead characteristics (CPU, memory, network bandwidth) under representative CI/CD workloads. The reproducibility target includes deployment templates, measurement procedures, and benchmark scenarios that will enable independent validation of findings. The constraint documentation and mitigation patterns established in this chapter will inform the architectural decisions and evaluation methodology to be applied in subsequent research phases.

2.9 Summary and Implications for Subsequent Research

This chapter has established the methodological foundation for systematic literature review and synthesized existing research on container observability, with particular attention to deployment assumptions, performance characteristics, and the alignment between documented practices and the operational constraints of privilege-restricted CI/CD environments. The reviewed literature demonstrates that contemporary observability systems achieve rich telemetry collection through diverse instrumentation techniques (host-based kernel probes, sidecar proxies, application-level tracing libraries), each exhibiting distinct privilege requirements, performance overheads, and applicability boundaries. While kernel-level eBPF mechanisms offer substantial efficiency advantages, their dependence on host access and recent

kernel features limits transferability to heterogeneous or restricted platforms. Sidecar patterns provide deployment flexibility but introduce resource multipliers and microarchitectural contention. Application-level instrumentation enables fine-grained observability at the cost of code modification and cross-platform compatibility challenges.

The comparative analysis reveals that much of the published research presumes operational capabilities unavailable in constrained CI/CD deployments: administrative host access, orchestrator-mediated resource management, and the ability to modify or instrument application code. Evaluation practices, while methodologically diverse, predominantly target long-lived microservice workloads rather than ephemeral task-based executions characteristic of CI/CD pipelines. Metrics such as MTTD and MTTR, though frequently invoked, lack standardized definitions suitable for comparing observability interventions across heterogeneous systems. These findings underscore the necessity of adapting existing instrumentation techniques and evaluation methodologies to the specific constraints and workload characteristics of on-premise, multi-tenant CI/CD infrastructure.

The mapping presented in Section 2.8 clarifies how the synthesis in this chapter informs the structure of subsequent work. Each research question is systematically grounded in the reviewed evidence: RQ1 (Section 2.8.1) establishes the privilege constraints documented across instrumentation approaches; RQ2 (Section 2.8.2) synthesizes performance trade-offs and identifies gaps in heterogeneous environment guidance; RQ3 (Section 2.8.3) documents operational constraints and methodological limitations in existing evaluation practices. The identified gaps (particularly the absence of empirical studies characterizing non-privileged telemetry collection in CI/CD contexts, limited guidance on post hoc correlation for ephemeral workloads, and methodological gaps in controlled evaluation frameworks) directly motivate the design, implementation and evaluation phases. Future chapters will leverage the documented privilege constraints and deployment patterns to specify a container-first reference architecture, implement a reproducible prototype that operationalizes feasible instrumentation approaches within identified boundaries, and evaluate the solution using standardized measurement procedures against the evidence base established here. Collectively, the evidence synthesized in this chapter establishes the research context and bridges the gap between general-purpose observability research and the specific requirements of constrained CI/CD observability.

Chapter 3

Project Planning and Methodology

This chapter presents how the dissertation project will be carried out during execution (January–June 2026). Building on the problem framing in Chapter 1 and state of the art in Chapter 2, the focus is on methodological approach, planned artifacts, evaluation activities, skills development, schedule, and risk management.

3.1 Objectives and Scope Alignment

The project objectives and scope defined in Chapter 1 remain aligned with findings synthesized in Chapter 2. No scope changes are planned. The work focuses on execution strategy to deliver planned artifacts and produce defensible evidence.

3.2 Methodological Approach

The project is planned as an engineering-oriented research effort centered on the design, implementation, and evaluation of a container-first observability solution that operates under the practical constraints identified in Chapter 1. The chosen approach is pragmatic and iterative: requirements and constraints inform successive design decisions, which are validated through controlled experimentation and practitioner-oriented feedback.

This section describes the methodological structure guiding the dissertation execution. The subsections specify the concrete artifacts to be produced, the development process ensuring traceability from constraints to design decisions, and the evaluation strategy for generating defensible empirical evidence. Together, these components establish how the research moves from problem framing and literature synthesis to implemented solutions and validated outcomes.

3.2.1 Planned Artifacts

The project will produce a set of artifacts intended to be directly auditable and reusable:

- **A1 – Reference architecture:** a documented architecture describing components, deployment topology, data flows, and correlation mechanisms suitable for containerized environments with limited privileges.
- **A2 – Prototype implementation:** a working prototype implementing the reference architecture in a staging environment, including configuration and operational procedures.

- **A3 – Automation and reproducibility assets:** scripts and configuration files to provision, deploy, and execute the evaluation campaign in a repeatable manner.
- **A4 – Evaluation protocol and dataset:** an experimental plan (scenarios, workloads, baselines, metrics) and the resulting collected data, with clear provenance.
- **A5 – Documentation package:** technical documentation for the architecture, deployment, and evaluation procedures, prepared to support both thesis writing and oral defense.

3.2.2 Development Process and Traceability

The development process ensures traceability from constraints to design decisions and from design decisions to evaluation evidence:

- **Constraint-to-design mapping:** constraints (e.g., limited privileges, heterogeneous execution environments) are mapped to explicit architectural and implementation decisions.
- **Iterative refinement:** the architecture and prototype are developed in short iterations, each concluding with a review of the produced artifact increment and the update of open issues.
- **Configuration management:** all artifacts are maintained under version control, including deployment manifests, configuration, and evaluation scripts.
- **Acceptance criteria:** each major deliverable is associated with objective acceptance criteria (e.g., successful deployment steps, reproducible execution of an experiment, completeness of documentation).

3.2.3 Planned Evaluation Strategy

Evaluation is planned to produce evidence that the proposed solution is feasible under the stated constraints and provides measurable utility when compared with a defined baseline. The evaluation design is bounded to what can be executed within the dissertation schedule and the available infrastructure.

Evaluation dimensions. The evaluation covers:

- **Feasibility under constraints:** whether the deployment and operation steps are compatible with the assumed privilege and platform limitations.
- **Operational overhead:** resource consumption and performance overhead introduced by the solution (measured using a defined workload and baseline).
- **Observability usefulness:** the extent to which the collected telemetry supports diagnosing representative failure and degradation scenarios.
- **Maintainability and operability:** the clarity of configuration, deployment, and troubleshooting procedures as captured in the documentation.

Experimental design (planning level). The experimental campaign is structured as follows:

- **Baselines:** at least one baseline configuration is defined to enable comparative measurements (e.g., no additional observability components, or minimal instrumentation).

- **Workloads and scenarios:** a small set of representative workloads and fault/degradation scenarios are specified to exercise logging, metrics, and tracing paths.
- **Data collection:** procedures are documented to collect resource metrics, request/-trace metadata, and logs, ensuring consistent time windows and environment configuration.
- **Analysis:** the analysis plan defines how measurements are summarized and compared, and how qualitative observations are recorded.

Practitioner feedback (planning level). Where access and availability permit, the project includes structured feedback from practitioners (e.g., hosting organization stakeholders) using an interview or review protocol. The protocol defines eligibility, consent, the question guide, and how responses are summarized (e.g., thematic coding at a coarse granularity) to support defensible qualitative claims.

3.3 Skills Development Plan (PREPD)

This section defines a skills development plan aligned with the project phases. The plan identifies the competencies required for the dissertation, the current diagnosis, development actions, and objective assessment criteria.

The competency framework organizes required capabilities into three domains: observability engineering in constrained environments, experimental evaluation with attention to validity, and technical communication for both written and oral presentation. The subsection that follows diagnoses current and target competency levels, specifies planned development actions mapped to project phases, and defines objective criteria for assessing whether competency targets have been achieved.

3.3.1 Competency Diagnosis

Table 3.1: Competency Diagnosis and Targets

Competency	Current Level	Target Level	Relevance
C1: Observability engineering in containers	Intermediate: operational familiarity with containers and basic telemetry tooling	Advanced: design and deploy a constrained, production-oriented telemetry stack	Core competency to implement A1–A3
C2: Experimental evaluation and validity	Foundational: understanding of evaluation concepts; limited campaign execution	Intermediate: define baselines, run experiments, analyze results, discuss threats	Required to produce A4 and defensible evidence
C3: Technical communication and stakeholder interaction	Intermediate: academic writing; limited structured practitioner engagement	Advanced: concise documentation, structured feedback collection, synthesis	Required for A5 and oral defense readiness

3.4 Project Planning and Schedule

The project plan is organized into phases with explicit deliverables and dependencies. The sequencing is designed to reduce risk by validating feasibility early (technology selection and staging deployment) before committing to extended evaluation work.

Table 3.2: Skills Development Actions Mapped to Project Phases

Comp.	Planned Development Actions (by phase)	Assessment Criteria
C1	**P1–P2:** deep-dive on candidate tooling and deployment patterns; implement staging deployment; document operational procedures. **P3:** iterate architecture and prototype based on deployment feedback and constraint compliance.	Prototype deploys from documented procedure; architecture decisions are traceable to constraints; operational runbook is complete.
C2	**P2:** define evaluation protocol (metrics, baselines, scenarios, data collection). **P3–P4:** execute measurement campaign; perform comparative analysis; document threats to validity.	Evaluation protocol is reproducible; dataset provenance is documented; analysis includes baselines and validity discussion.
C3	**P1:** consolidate PREPD writing and structure. **P3–P5:** produce technical documentation; prepare interview/review protocol; synthesize practitioner feedback where applicable.	Documentation supports independent replication; feedback protocol is structured; writing and oral preparation are consistent and coherent.

This section presents the temporal organization of dissertation activities from January through June 2026. The subsections describe the phase structure with associated activities and deliverables, define milestones and their dependencies to support periodic progress assessment, and provide a visual timeline through a Gantt chart. This structure enables systematic tracking of progress and early identification of schedule risks.

3.4.1 Phases, Activities, and Deliverables

Table 3.3: Planned Phases, Activities, and Deliverables (January–June 2026)

Phase	Period	Key Activities	Main Deliverables
P1	Jan	Project setup; consolidation of requirements and constraints; definition of acceptance criteria and logs (issues/risks).	Execution plan; project logs; refined acceptance criteria.
P2	Jan–Feb	Technology assessment and selection; proof-of-concept validations; definition of the architecture decision record structure.	Selection matrix and justification; initial architecture decisions.
P3	Feb–Mar	Reference architecture specification; prototype implementation; staging deployment; automation and reproducibility assets.	A1–A3 (architecture, prototype, automation); deployment runbook.
P4	Apr–May	Evaluation campaign execution: baseline definition, scenarios, measurements, data collection, preliminary analysis.	A4 (evaluation protocol and dataset); preliminary analysis notes.
P5	May–Jun	Consolidated analysis; practitioner feedback (when feasible); thesis writing and revision cycles; defense preparation.	A5 (documentation package); completed thesis manuscript.

3.4.2 Milestones and Dependencies

Milestones are defined to support periodic assessment and early detection of schedule risk. Key dependencies are: (i) technology selection before architecture freeze, (ii) staging deployment before extended evaluation, and (iii) availability of infrastructure and stakeholders

before practitioner feedback.

Table 3.4: Planned Milestones and Acceptance Focus

ID	Planned Date	Milestone	Acceptance Focus
M1	End Jan	Technology selection completed	Selection rationale documented; fallbacks identified; feasibility validated via proof-of-concept.
M2	Mid Mar	Staging prototype deployed	Deployment is repeatable; core telemetry pipeline works end-to-end; runbook is usable.
M3	End Apr	Evaluation protocol frozen	Baselines and scenarios defined; collection procedures documented; validity considerations recorded.
M4	End May	Measurement campaign executed	Dataset collected with provenance; key analyses executed; issues and limitations captured.
M5	Mid Jun	Thesis ready for submission	Chapters integrated; evidence traceable to artifacts; revisions incorporated; defense preparation completed.

3.4.3 Gantt Chart

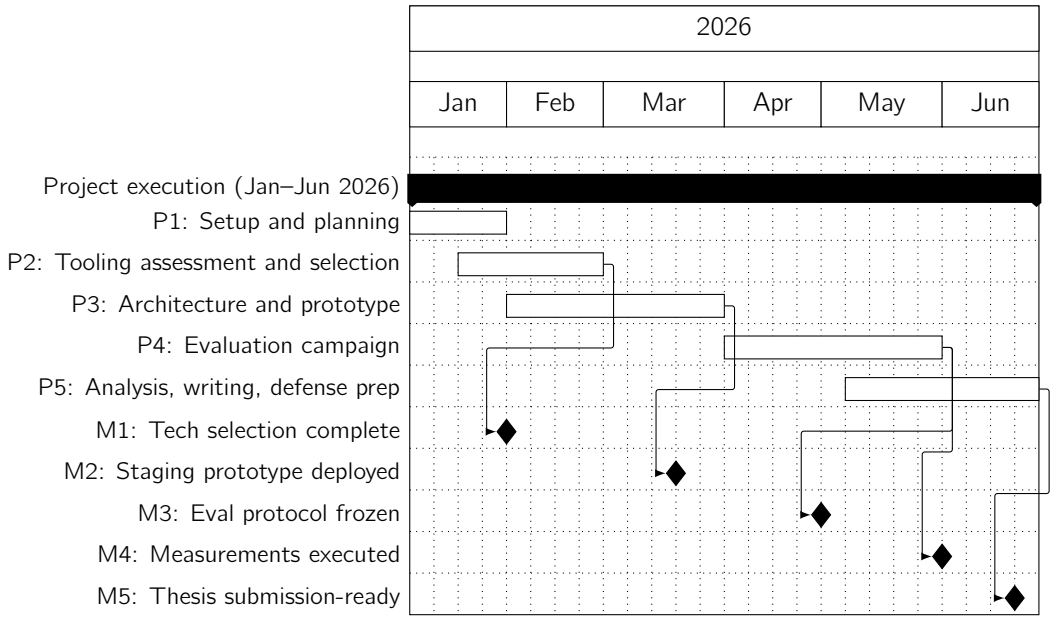

Figure 3.1: Project schedule overview (Gantt chart; January–June 2026)

3.5 Project Monitoring and Management

Project monitoring is structured to ensure timely control of scope, schedule, and quality:

- **Governance and roles:** responsibilities are split between the student (execution and reporting), the academic advisor (scientific guidance and quality control), and the

hosting institution supervisor (infrastructure coordination and practitioner feedback facilitation when applicable).

- **Cadence:** meetings are held regularly (e.g., bi-weekly with the academic advisor; weekly or as-needed with the hosting supervisor) to review progress against the plan and to unblock dependencies.
- **Progress tracking:** a lightweight project log is maintained, including tasks, deliverables, open issues, decisions, and risk status.
- **Change control:** any scope adjustment is documented with rationale, impact analysis (schedule/risk), and approval by the advisor.
- **Quality control:** each milestone includes an acceptance review focused on reproducibility, completeness of documentation, and alignment with constraints.

3.6 Risk Management

Risk management is continuous and integrated with monitoring. Risks are identified early, assessed by probability and impact, tracked through indicators, and mitigated through preventive actions. When mitigation is insufficient, a contingency plan is activated to preserve the ability to complete the dissertation within the time constraints.

To keep the assessment consistent and auditable, this project uses a lightweight qualitative scale for both *Impact* and *Probability*. Impact captures the consequence to delivery (scope, schedule, or validity of results); probability captures the likelihood of occurrence within the January–June 2026 execution window. Monitoring indicators are defined as observable signals with thresholds (where feasible) that trigger mitigation or contingency actions.

Table 3.5: Risk Rating Scales and Measurement Approach

Dimension	Class	Operational definition (how it is measured)
Impact	H	Prevents delivering a core artifact (A1–A4) or forces a major scope reduction; or causes a schedule slip > 2 weeks on a milestone.
	M	Degrades evidence quality or requires rework; or causes a schedule slip of ~ 1–2 weeks.
	L	Localized issue with limited rework; schedule impact < 1 week.
Probability	H	Likely (> 60%): already observed signals or strong prior evidence in similar environments.
	M	Possible (20–60%): plausible given constraints; may occur depending on approvals/changes.
	L	Unlikely (< 20%): would require unusual conditions or multiple contributing events.

The risk register is reviewed at least monthly and at each milestone review. For high-impact risks, mitigation status and contingency readiness are reported in progress updates.

Table 3.6: Risk Register (technical, schedule, and external risks)

ID	Type	Risk Description	Impact	Prob.	Monitoring Indicators	Mitigation / Contingency
R1	Tech	Tooling or deployment does not comply with privilege/platform constraints, or is not reliable in the target environment.	H	M	Proof-of-concept fails to deploy >2 times; required capability missing; unstable telemetry pipeline.	Run early proof-of-concept; keep fallback options; freeze a minimal viable architecture and reduce scope if needed.
R2	Tech	Telemetry pipeline quality and/or overhead is insufficient for the planned diagnosis scenarios.	H	M	Trace/log correlation breaks; missing identifiers; CPU/memory overhead exceeds agreed bounds; latency regression beyond baseline tolerance.	Validate correlation end-to-end early; tune configuration and sampling; prioritize fewer scenarios with stronger instrumentation; bound evaluation and document limitations.
R3	Schedule	Infrastructure access, deployment approvals, or environment instability delays evaluation activities.	H	M	Access not granted by planned date; blocked deployments; long coordination cycles; repeated environment outages.	Use staging as primary target; parallelize protocol design and writing; maintain schedule buffers; adjust campaign size to preserve M3–M5.
R4	Schedule	Writing and review iteration compresses due to technical delays, reducing dissertation quality.	H	M	Missed weekly writing checkpoint; advisor review windows missed; late integration of chapters/evidence.	Enforce weekly writing output; draft chapters incrementally; lock review dates; de-scope non-essential work to protect submission readiness.
R5	External	Limited stakeholder availability or ecosystem/policy changes reduce feedback feasibility or impose new constraints.	M	M	Low response rate; repeated rescheduling; new restrictions; deprecations impacting selected components.	Recruit early; allow asynchronous reviews; complement with advisor/expert review; choose conservative components; document assumptions and adapt plan.

Chapter 4

Conclusion and Next Steps

This chapter concludes the Preparation of Dissertation (PREPD) phase. The preparation work presented in this document establishes the foundation for thesis execution. No results, findings, or experimental outcomes are presented, as empirical work will be conducted during the subsequent dissertation phase.

4.1 Synthesis of Preparation Work

Chapter 1 framed the research problem: observability in containerized CI/CD platforms under non-privileged access constraints and heterogeneous infrastructure. The problem arises from the gap between standard monitoring assumptions (host administrative access, privileged containers, cloud-managed platforms) and organizational realities where such capabilities are unavailable. The central research question examines how an integrated observability framework can be designed and evaluated to improve reliability and reduce incident response times under these constraints.

Chapter 2 synthesized the state of the art through systematic literature review following PRISMA guidelines. The review identified relevant observability patterns, instrumentation techniques, and evaluation approaches documented in prior work. The synthesis established that while observability frameworks for cloud-native environments are extensively documented, approaches addressing non-privileged containerized deployments in heterogeneous infrastructure remain underexplored in the literature.

Chapter 3 defined the project methodology and execution strategy. The approach centers on engineering-oriented research involving design, implementation, and evaluation of a container-first observability solution. Planned artifacts include reference architecture, prototype implementation, automation assets, evaluation protocol, and documentation package. The evaluation strategy specifies assessment dimensions: feasibility under constraints, operational overhead, observability usefulness, and maintainability.

4.2 Transition to Thesis Execution

The thesis execution phase will implement and evaluate the planned solution. Immediate next steps include:

- Complete technology selection based on compatibility with identified constraints, documented in assessment matrix with explicit rationale.

- Design reference architecture specifying components, deployment topology, data flows, and correlation mechanisms.
- Implement prototype in staging environment, producing deployment templates, configuration artifacts, and operational procedures.
- Execute evaluation campaign comparing baseline and instrumented deployments under defined workloads with controlled fault injection.
- Collect and analyze measurement data, prepare dissertation chapters documenting implementation, evaluation findings, and implications.

Validation targets defined in project planning include quantitative metrics (MTTD, MTTR), telemetry pipeline throughput verification, alert precision and detection coverage assessment via fault injection, and instrumentation overhead quantification with an operational target below 5% where feasible. These targets are operationalized through specific measurement procedures and acceptance criteria documented in the evaluation protocol. Analysis is restricted to tested contexts without overgeneralization.

The work documented in this PREPD confirms readiness to proceed with thesis execution. All preparatory activities required to initiate implementation and evaluation have been completed.

Bibliography

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, omega, and kubernetes. *Queue*, *14*(1), 70–93.

Correia, M., Oliveira, W., & Cecílio, J. (2023). Monintainer: An orchestration-independent extensible container-based monitoring solution for large clusters. *Journal of Systems Architecture*, *145*, 103035. https://doi.org/10.1016/j.sysarc.2023.103035

Datadog. (2023). *Hybrid Cloud Monitoring: Best Practices* [Technical blog and whitepapers discussing agent deployment models and observability across hybrid infrastructure]. Retrieved December 15, 2024, from https://www.datadoghq.com/blog/

Dinga, M., Malavolta, I., Giamattei, L., Guerriero, A., & Pietrantuono, R. (2023). An Empirical Evaluation of the Energy and Performance Overhead of Monitoring Tools on Docker-Based Systems. *Service-Oriented Computing*, 181–196. https://doi.org/10.1007/978-3-031-48421-6_13

Dynatrace. (2023). *Enterprise Observability Architecture* [Technical documentation on agent deployment in customer-managed environments and multi-tenant observability]. Retrieved December 15, 2024, from https://docs.dynatrace.com/docs

Elastic. (2024). *Elastic Observability: Architecture and Deployment* [Documentation describing hybrid push-pull patterns, agent architecture, and enterprise deployment considerations]. Retrieved December 15, 2024, from https://www.elastic.co/guide/en/observability/current/index.html

Forsgren, N., Smith, D., Humble, J., & Frazelle, J. (2024). The 2024 state of devops report. *DORA (DevOps Research and Assessment)*.

GitLab. (2024). *GitLab CI/CD: Observability and Monitoring* [Platform documentation describing log-based troubleshooting and pipeline monitoring capabilities]. Retrieved December 15, 2024, from https://docs.gitlab.com/ee/ci/

Grafana Labs. (2024). *Grafana Loki: Log Aggregation System* [Technical documentation describing log aggregation architecture, indexing strategies, and storage trade-offs]. Retrieved December 15, 2024, from https://grafana.com/docs/loki/latest/

Großmann, M., & Klug, H. (2017). Monitoring Container Services at the Network Edge. *2017 29th International Teletraffic Congress (ITC 29)*, *1*, 130–133. https://doi.org/10.23919/ITC.2017.8064348

Haddaway, N. R., Page, M. J., Pritchard, C. C., & McGuinness, L. A. (2022). *PRISMA2020: An R package and Shiny app for producing PRISMA 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis* (Version 1.1.1). Zenodo. https://doi.org/10.5281/zenodo.5082518

Hasanth, K. M., & Nadig, D. (2024). Evaluating the Performance of Sidecar-based and Sidecarless Cloud-native Service Mesh Solutions. *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, 1–6. https://doi.org/10.1109/ANTS63515.2024.10897788

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105.

IEEE Xplore Full-Text PDF: (n.d.).

Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems. (n.d.). https://doi.org/10.1145/3680256.3721316

Janecek, M., Ezzati-Jivan, N., & Azhari, S. V. (2021). Container Workload Characterization Through Host System Tracing. *2021 IEEE International Conference on Cloud Engineering (IC2E)*, 9–19. https://doi.org/10.1109/IC2E52221.2021.00015

Jenkins Project. (2024a). *Jenkins OpenTelemetry Plugin* [Plugin documentation for distributed tracing integration in Jenkins pipelines]. Retrieved December 15, 2024, from https://plugins.jenkins.io/opentelemetry/

Jenkins Project. (2024b). *Jenkins: Monitoring and Observability* [Official documentation on pipeline observability, logging practices, and troubleshooting workflows]. Retrieved December 15, 2024, from https://www.jenkins.io/doc/book/system-administration/monitoring/

Kannan, P. G., Gupta, S. M., Behl, D., Raichstein, E., & Takvorian, J. (2024). Designing a lightweight network observability agent for cloud applications. *Passive and Active Measurement*, 262–276. https://doi.org/10.1007/978-3-031-56249-5_11

Karkan, T. M. (n.d.). Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure.

Levin, J. (2020). ViperProbe: Using eBPF Metrics to Improve Microservice Observability.

Liu, H., Zhang, J., Shan, H., Li, M., Chen, Y., He, X., & Li, X. (2019). JCallGraph: Tracing Microservices in Very Large Scale Container Cloud Platforms. *Cloud Computing – CLOUD 2019*, 287–302. https://doi.org/10.1007/978-3-030-23502-4_20

Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux J.*, *2014*(239).

Norgren, E. (n.d.). OPTIMIZING DISTRIBUTED TRACING OVERHEAD IN A CLOUD ENVIRONMENT WITH OPENTELEMETRY.

OpenTelemetry Authors. (2024). *OpenTelemetry Collector: Architecture and Deployment Patterns* [Official documentation describing collector deployment patterns, configuration options, and operational considerations]. Retrieved December 15, 2024, from https://opentelemetry.io/docs/collector/

Pappula, K. K., Anasuri, S., & Rusum, G. P. (2021). Building observability into full-stack systems: Metrics that matter. *International Journal of Emerging Research in Engineering and Technology*, *2*(4), 48–58.

Pi, A., Chen, W., Zhou, X., & Ji, M. (2018). Profiling distributed systems in lightweight virtualized environments with logs and resource metrics. *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*, 168–179. https://doi.org/10.1145/3208040.3208044

Prometheus Authors. (2024). *Prometheus: Operational Aspects* [Operational guidelines for pull-based metric collection, service discovery, and deployment patterns]. Retrieved December 15, 2024, from https://prometheus.io/docs/introduction/overview/

Raith, P., Rausch, T., Prüller, P., Furutanpey, A., & Dustdar, S. (2022). An End-to-End Framework for Benchmarking Edge-Cloud Cluster Management Techniques. *2022 IEEE International Conference on Cloud Engineering (IC2E)*, 22–28. https://doi.org/10.1109/IC2E55432.2022.00010

Ramachandran, G. S., McDonald, L., & Jurdak, R. (2023). FUSE: Fault Diagnosis and Suppression with eBPF for Microservices. *Service-Oriented Computing*, 243–257. https://doi.org/10.1007/978-3-031-48421-6_17

Sahu, P., Zheng, L., Bueso, M., Wei, S., Yadwadkar, N. J., & Tiwari, M. (2023, October). Sidecars on the Central Lane: Impact of Network Proxies on Microservices. https://doi.org/10.48550/arXiv.2306.15792

Salcedo-Navarro, A., Garcia-Pineda, M., & Gutiérrez-Aguado, J. (2025). K8sidecar: A Modular Kubernetes Chain of Sidecar Proxies for Microservices and Serverless Architectures. *Software: Practice and Experience*, *55*(8), 1305–1319. https://doi.org/10.1002/spe.3423

Sandberg, F. (n.d.). Evaluating OpenTelemetry's Impact on Performance in Microservice Architectures.

Sharma, B. (2023, April). *IMPROVING MICROSERVICES OBSERVABILITY IN CLOUD-NATIVE INFRASTRUCTURE USING EBPF* [Thesis]. Purdue University Graduate School. https://doi.org/10.25394/PGS.22682623.v1

Sharma, B., & Nadig, D. (2024). eBPF-Enhanced Complete Observability Solution for Cloud-native Microservices. *ICC 2024 - IEEE International Conference on Communications*, 1980–1985. https://doi.org/10.1109/ICC51166.2024.10622329

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). Dapper, a large-scale distributed systems tracing infrastructure. *Google Technical Report*.

Soldani, D., Nahi, P., Bour, H., Jafarizadeh, S., Soliman, M. F., Di Giovanna, L., Monaco, F., Ognibene, G., & Risso, F. (2023). eBPF: A New Approach to Cloud-Native Observability, Networking and Security for Current (5G) and Future Mobile Networks (6G and Beyond). *IEEE Access*, *11*, 57174–57202. https://doi.org/10.1109/ACCESS.2023.3281480

Tan, Y., Quan, S., Zhu, Z., & Gao, S. (2023). Research on A Lightweight Service Mesh Architecture Using eBPF. *2023 4th International Conference on Computer Engineering and Application (ICCEA)*, 52–55. https://doi.org/10.1109/ICCEA58433.2023.10135394

Taylor, T., Araujo, F., & Shu, X. (2020). Towards an Open Format for Scalable System Telemetry. *2020 IEEE International Conference on Big Data (Big Data)*, 1031–1040. https://doi.org/10.1109/BigData50022.2020.9378294

TraceWeaver: Distributed Request Tracing for Microservices Without Application Modification. (n.d.). https://doi.org/10.1145/3651890.3672254

Usman, M., Ferlin, S., Brunstrom, A., & Taheri, J. (2023). DESK: Distributed Observability Framework for Edge-Based Containerized Microservices. *2023 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, 617–622. https://doi.org/10.1109/EuCNC/6GSummit58263.2023.10188344

Volz, J. (2016). Prometheus: Monitoring at soundcloud. *SoundCloud Developers Blog*.